



BOOTCAMP FRONTEND & REACT · SEMANA 2

Responsive

Clase 5 — Unidades, media queries, variables CSS, modo oscuro y transiciones

4 h 30 · L·M·V

Proyecto TechCart

STEERCLAVE

PLAN DE HOY

Agenda

1. Repaso y tareas

~35'

Las dos difíciles, resueltas juntos

2. Unidades

~20'

%, em, rem, vh/vw · el N22 del juego

3. Media queries

~58'

Auditoría móvil · imagen fluida · 2 breakpoints

4. Mobile-first

~12'

min-width: la otra dirección

5. Variables CSS

~26'

La paleta con nombre (design tokens)

6. Modo oscuro

~26'

prefers-color-scheme + tokens

7. Transiciones

~30'

Micro-interacciones con transform

8. Franja de beneficios

~25'

Sección nueva: integra todo

**Receso de 15 min** a mitad de camino · cierre con 5 tareas · bonus si sobra: @keyframes

Repaso exprés · Clase 4

Dos de memoria

¿Qué significa `grid-column: 1 / -1`?

¿Qué hace `justify-self` que no hace `justify-items`?

LA DEUDA DESDE LA CLASE 3 · 1REM = 16PX · 2REM = 32PX · 100VH = PANTALLA COMPLETA

Las formas de medir

px fijo: mide eso, pase lo que pase

% relativo al PADRE (su contenedor)

em relativo a la letra donde estás · se ANIDA

rem relativo a la RAÍZ (html = 16px)

vh · **vw** % del alto / ancho de la PANTALLA

padre con letra de 12px

 6em = 72px

 6rem = 96px

padre con letra de 20px

 6em = 120px ← cambió

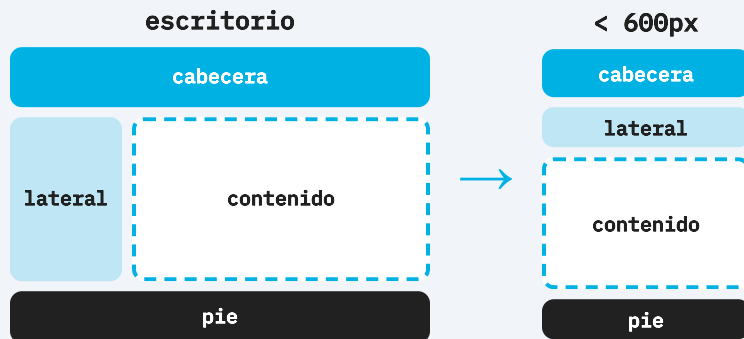
 6rem = 96px ← igual

em depende de dónde está parado · **rem** siempre mira la raíz. Ante la duda: **rem**.

LA MISMA PÁGINA, EN CUALQUIER PANTALLA · REQUIERE EL META VIEWPORT (LO TENEMOS DESDE LA CLASE 1)

@media — reglas con condición

```
/* "si la pantalla mide 600px o
menos,
  aplica esto" */
@media (max-width: 600px) {
  header { flex-direction:
column; }
}
/* fuera de la condición,
  el bloque NO existe */
```



LA AUDITORÍA ENCONTRÓ 6 FALLAS · UN BLOQUE @MEDIA TACHA 4 · LAS OTRAS 2 SON DE USTEDES

El plan de ataque

- 1 La cabecera se apila — `flex-direction: column`
- 2 El plano, re-dibujado a UNA columna — `grid-template-areas` apilado
- 3 La deuda de la destacada, pagada — `grid-column: auto`
- 5 El título, a escala — `font-size: 1.5rem`
- 4·6 **Ejercicio:** el formulario (Grid) y el footer (Flexbox), al celular — dos sistemas, un patrón

NO HAY NÚMEROS MÁGICOS: EL CORTE SALE DE MIRAR DÓNDE SE ROMPE EL CONTENIDO

Breakpoints: de mayor a menor

celular ≤ 600px · todo apilado	tablet 601 – 900px · lateral angosta	escritorio > 900px · la hoja base
--	--	---

```
@media (max-width: 900px) { /* tablet – va ARRIBA */ }  
@media (max-width: 600px) { /* celular – va DEBAJO */ }
```

A 500px se cumplen **las dos**: en lo que choquen, gana la que está **más abajo** — la cascada de siempre.

MAX-WIDTH = TECHO ("HASTA") · MIN-WIDTH = PISO ("DESDE") · TAILWIND ES MOBILE-FIRST DE FÁBRICA

Dos direcciones, mismo resultado

desktop-first – lo de hoy: RESTAR

```
/* la base describe el ESCRITORIO
*/
body { grid-template-columns:
  220px 1fr; }

/* ...y hacia abajo se quita */
@media (max-width: 600px) {
  body { grid-template-columns:
    1fr; }
}
```

mobile-first – al empezar de cero: SUMAR

```
/* la base describe el CELULAR */
body { grid-template-columns: 1fr;
}

/* ...y hacia arriba se agrega */
@media (min-width: 900px) {
  body { grid-template-columns:
    220px 1fr; }
}
```

DESIGN TOKENS: DECLARO UNA VEZ, USO MIL VECES, CAMBIO EN UN SOLO LUGAR

Variables CSS: la paleta con nombre

```
/* se declaran en la raíz... */
:root {
  --marca: #0066cc;
  --fondo: #f5f5f7;
  --tarjeta: #fff;
}

/* ...y se usan con var() */
h1 { color: var(--marca); }
button { background: var(--marca); }
a { color: var(--marca); }
```

Cambias **una línea** (--marca) y cambia **toda la tienda**: título, enlaces, botones, bordes.

⚠ Un var() mal escrito falla **en silencio**: sin error en consola. En DevTools sale **tachado**.

MISMOS NOMBRES, OTROS VALORES: NI UNA REGLA DE LAYOUT SE TOCA

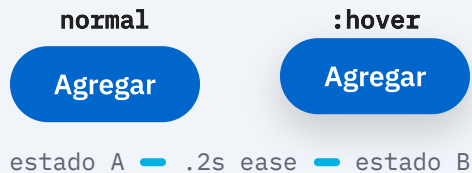
prefers-color-scheme

```
/* la misma paleta, de noche */  
@media (prefers-color-scheme: dark)  
{  
  :root {  
    --fondo: #111114;  
    --tarjeta: #1d1d1f;  
    --texto: #f5f5f7;  
    --marca: #3d9bff;  
  }  
}
```



MICRO-INTERACCIONES: CORTAS (0.1–0.3S) Y CON PROPÓSITO · TRANSFORM NO EMPUJA A LOS VECINOS

transition — no saltes: anima



Regla de oro: la transition se escribe en el **estado BASE**, no en el :hover — si no, la ida es suave y la vuelta salta.

SECCIÓN NUEVA, CERO CONCEPTOS NUEVOS: UN EXAMEN DISFRAZADO · HEREDA TARJETA, HOVER Y MODO OSCURO GRATIS

La franja de **beneficios**



Resumen

%, em, rem, vh/vw

1rem = 16px · ante la duda, rem

@media (max-width)

breakpoints 900/600, de mayor a menor

width:100% + object-fit

la imagen fluida jamás rompe el layout

:root + var()

tokens: una línea cambia toda la tienda

prefers-color-scheme

modo oscuro = mismos tokens, otros valores

transition + transform

en el estado base · sin empujar vecinos

PARA CASA · LA QUE NADIE LOGRE, LA ARMAMOS JUNTOS LA PRÓXIMA

5 tareas

FÁCIL

Categorías en fila: en el @media de 600px, la lista de la lateral con Flexbox + gap.

INTERMEDIO

Modo oscuro completo: los inputs del formulario siguen blancos — arréglalo con los tokens.

INTERMEDIO

Tokens de forma: --radio y --espacio; cambia --radio a 0 y mira tu tienda "cuadrada".

DIFÍCIL

Zoom de producto: scale(1.05) en la imagen sin salirse del marco (overflow: hidden).

DIFÍCIL

Cabecera pegajosa: position: sticky + top: 0 — ojo con z-index y el body-grid.

Próxima clase

Clase 6 — Tailwind CSS

Cuatro clases escribiendo CSS a mano: cajas, Flexbox, Grid, responsive, tokens. Tailwind lo **comprime** en utilidades con nombre —**flex, grid, gap-4, dark:**— y es **mobile-first de fábrica**: el min-width de hoy es su idioma nativo. Como ustedes ya escribieron el CSS real, van a entender qué hace cada utilidad por dentro.

Pista para casa: entra a tailwindcss.com, busca "flex" o "grid-template-columns" en su documentación y compara con lo que escribimos.

BLOQUE BONUS · SOLO SI SOBRA TIEMPO · DE REACCIONAR (TRANSITION) A VIVIR (ANIMATION)

@keyframes — coreografía de pasos

```
.badge {  
  animation: latido 1.4s ease infinite;  
}  
  
@keyframes latido {  
  0% { transform: scale(1); }  
  50% { transform: scale(1.08); }  
  100% { transform: scale(1); }  
}
```

